Functions: (user Defined).

input → [≡] → Reusable

→ return OUTPUT / NO OUTPUT

void / int / f / return Data Type — Function Name — formal

```
int addnum( int i, int j )
{
    return i + j    'Hello'
}.
```

↳ Parameters / arguments

```
def <function Name> (args):
    ≡
    return value | variable.
```

```
def Test_function( x, y, z ):
    return 3*x + 4*y + 5*z
```

define — formal Params

Call: ← Test_function(10, 20, 30) ↳ actual Param

```
def. Test_function( x, y, z ):
    return 3*x + 4*y + 5*z
```

formal

Test_function(10, 20, 30) → Positional arguments

actual

30 + 80 + 150 → 260 ✓

Test_function(30, 10, 20) → 260.

30x 90 + 40 + 100 → 230

→ 260.

Test_function(z = 30, x = 10, y = 20) ✓

↳ Named arguments / Keyword arguments

read_csv(Sep, header, usecols, index_col, converters, skip_rows, skip_footer, n_rows, decimal, comments)

def Test_function(x, y, z=0)

- len(object) → int.
- id(object) → returns id.
- input('Prompt') → returns the user input

list {
- insert
- append
- extend

} functions

function(name, x, y, data, sep, usecols, index_col)

(data=, x=, y=, index_col=)

Positional, Keyword → valid. ✓

Keyword, Positional = Not valid.

list.pop(index = -1)

*args | **kwargs.

✓ iterables ✓

✓ Modules